

Bloxorz 3D

- Start with the game identity, not the technology.
- Say it is a Bloxorz-style 3D puzzle game.
- Mention Three.js/WebGL as the rendering base.
- Keep this short, then move to the actual feature list.

Simple line:

We built a playable 3D puzzle game where the player rolls a cuboid across a tile map until it stands on the goal.

What we built

- The project is not just one level.
- There are 8 official levels.
- Each level is checked for a shortest solution.
- The game also has a level editor and saved progress.
- Mention the main mechanics quickly:
fragile tiles, switches, bridges, teleport split mode.

Do not explain every mechanic yet. Slide 5 is for that.

Discrete game state

- The puzzle is stored as data, not guessed from the mesh.
- Main state: x, z, orientation.
- Standing block occupies one cell.
- Lying block occupies two cells.
- Legal moves are checked from occupied cells.
- The same occupied cells also decide falls, switches, and wins.

Key point:

The graphics are smooth, but the puzzle logic stays exact.

Movement and animation

- A key press first asks: is this move legal?
- If legal, the state changes to the next grid position.
- The mesh then animates toward that state.
- Position interpolation makes movement smooth.
- Quaternion rotation gives clean 3D rolling.
- Snapping after the move prevents floating-point drift.

This connects gameplay logic with transformation in graphics.

Level mechanics

- Normal tile: basic support tile.
- Fragile tile: unsafe when the block stands upright.
- Heavy switch: only works with upright block pressure.
- Soft switch: works when any occupied cell touches it.
- Bridge: changes the board during the level.
- Teleport: splits the block into two smaller cubes.
- Merge: split cubes must rejoin from valid adjacent cells.
- Goal: only accepts a standing full block.

Best explanation:

Each mechanic changes the move-validation graph.

Graphics pipeline

- Geometry: block, tiles, bridge meshes, particles.
- Model transform: object position, rotation, scale.
- View transform: camera position and direction.
- Projection: 3D scene mapped to perspective screen space.
- Rasterization: triangles become screen fragments.
- Fragment shading: lights and materials affect final color.
- Frame buffer: final pixels are written to the canvas.

Use this slide to show direct lecture connection.

Scene setup

- `PerspectiveCamera` gives depth and 3D viewing.
- `MeshPhysicalMaterial` handles lighting-based surfaces.
- `DirectionalLight` creates main shadows.
- `AmbientLight` keeps dark areas readable.
- Fog adds depth.
- SMAA reduces jagged edges.
- Bloom/vignette are postprocessing effects.
- The editor uses raycasting to convert mouse position into grid cells.

Important:

Do not claim custom block shaders. Three.js materials use WebGL shader programs internally.

Player input

- Keyboard controls handle desktop play.
- Touch controls support mobile play.
- The gameplay camera stays in smooth third-person follow mode.
- Camera framing updates from the active block or split cube.
- HUD shows moves, par, level, and hints.
- localStorage saves completed levels and best moves.

Good phrase:

The same game state drives desktop, mobile, and the third-person camera.

BFS solver

- BFS means breadth-first search.
- It explores moves level by level.
- First found solution is shortest.
- State includes position, orientation, bridge state, and split state.
- Hint uses the next move from BFS.
- check-levels verifies official pars.

Verified shortest paths:

1	First Steps	8
2	Over the Edge	7
3	Fragile Ground	6
4	Open Sesame	6
5	Heavy Duty	12
6	Split Decision	8
7	Twin Bridges	13
8	The Gauntlet	15

Command:

```
npm run check:levels
```

Code files

- `main.js`: app flow, menu transitions, level lifecycle.
- `grid.js`: tile objects, bridge toggling, visual highlights.
- `controls.js`: move rules, falling, split-block behavior.
- `levels.js`: level data, themes, pars.
- `solver.js`: BFS hint search.
- `editor.js`: custom level editing and saving.
- `scene.js`: WebGL renderer, procedural sky, fog, environment map.
- `camera.js`: third-person follow camera.
- `lighting.js`: ambient, hemisphere, directional lights, shadows.
- `particles.js`: dust, shatter, sparkles, win feedback.
- `scripts/check-levels.js`: automated solvability check.

If code is requested, start from `controls.js` and `grid.js`.

Demo plan

- Keep the live demo focused.
- First show level selection and saved progress.
- Play Level 1 to show normal movement.
- Jump to mechanics instead of solving every level.
- Show one fragile tile, one switch/bridge, and teleport split.
- Use one hint to connect back to BFS.
- Show editor only if time remains.

Timing rule:

Do not spend more than one minute on Level 1.